

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №6

по дисциплине: "Логика и основы алгоритмизации в инженерных задачах"

на тему: "Унарные бинарные операции над графами"

Выполнил:

Студент группы 24ВВВ3

Мингалёв М. А.

Принял:

к.н., доцент, Юрова О. В.

к.т.н., Деев М.В.

Пенза 2025

Цель

Изучить унарные и бинарные операции над графами

Теоретическая часть

Все унарные операции над графами можно объединить в две группы. Первую группу составляют операции, с помощью которых из исходного графа G_1 , можно построить граф G_2 с меньшим числом элементов. В группу входят операции удаления ребра или вершины, отождествления вершин, стягивание ребра. Вторую группу составляют операции, позволяющие строить графы с большим числом элементов. В группу входят операции расщепления вершин, добавления ребра.

Отождествление вершин. В графе G_1 выделяются вершины u, v . Определяют окружение Q_1 вершины u , и окружение Q_2 вершины v , вычисляют их объединение $Q = Q_1 \cup Q_2$. Затем над графом G_1 выполняются следующие преобразования:

- из графа G_1 удаляют вершины u, v ($H_1 = G_1 - u - v$);
- к графу H_1 присоединяют новую вершину z ($H_1 = H_1 + z$);
- вершину z соединяют ребром с каждой из вершин $w_i \in Q$ ($G_2 = H_1 + zw_i, i = 1, 2, 3, \dots$).

Стягивание ребра. Данная операция является операцией отождествления смежных вершин u, v в графе G_1 .

Наиболее важными бинарными операциями являются: объединение, пересечение, декартово произведение и кольцевая сумма.

Объединение. Граф G называется объединением или наложением графов G_1 и G_2 , если $V_G = V_1 \cup V_2$; $E_G = E_1 \cup E_2$ (рис. 1).

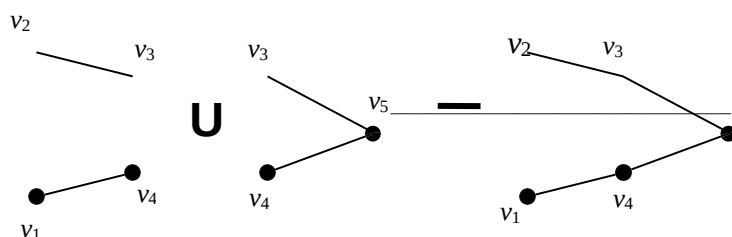


Рис. 1. Объединение графов G_1, G_2

Объединение графов G_1 и G_2 называется дизъюнктивным, если $V_1V_2 = \emptyset$. При дизъюнктивном объединении никакие два из объединяемых графов не должны иметь общих вершин.

Пересечение. Граф G называется пересечением графов G_1, G_2 , если $V_G = V_1V_2$ и $U_G = U_1U_2$ (рис.2). Операция "пересечения" записывается следующим образом: $G = G_1G_2$.

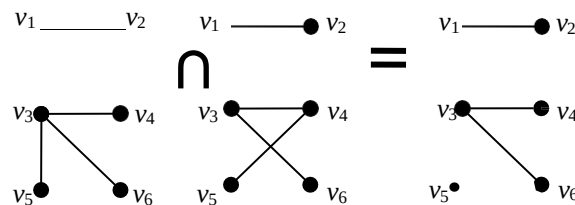


Рис.2. Пересечение графов G_1, G_2 .

Декартово произведение. Граф G называется декартовым произведением графов G_1 и G_2 если $V_G = V_1 \times V_2$ —декартово произведение множеств вершин графов G_1, G_2 , а множество ребер U_G задается следующим образом: вершины (z_i, v_k) и (z_j, v_l) смежны в графе G тогда и только тогда, когда $z_i = z_j (i = j)$, а v_k и v_l смежны в G_2 или $v_k = v_l (k = l)$, смежны в графе G_1 (см. рис.3).

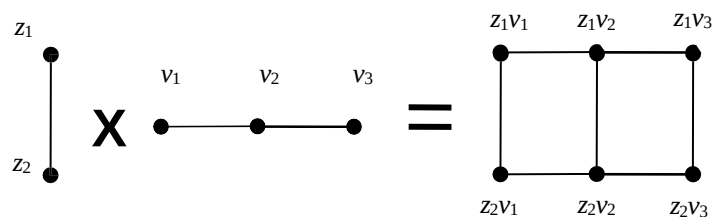


Рис. 3. Декартово произведение графов G_1, G_2

Кольцевая сумма графов представляет граф, который не имеет изолированных вершин и состоит из ребер, присутствующих либо в первом исходном графе, либо во втором. Кольцевая сумма определяется следующим соотношением: $G = G_1 \oplus G_2$ (рис.4).

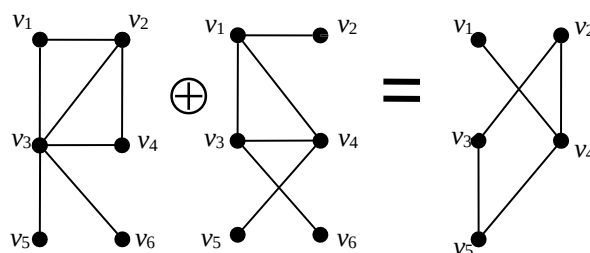


Рис.4. Кольцевая сумма графов G_1, G_2

Лабораторное задание:

1. Сгенерируйте (используя генератор случайных чисел) две матрицы M_1 , M_2 смежности неориентированных помеченных графов G_1 , G_2 . Выведите сгенерированные матрицы на экран.

2. Для матричной формы представления графов выполните операцию:

- а) отождествления вершин
- б) стягивания ребра
- в) расщепления вершины

Номера выбираемых для выполнения операции вершин ввести с клавиатуры.

Результат выполнения операции выведите на экран.

3. Для матричной формы представления графов выполните операцию:

- а) объединения $G = G_1 \cup G_2$
- б) пересечения $G = G_1 \cap G_2$
- в) кольцевой суммы $G = G_1 \oplus G_2$

Результат выполнения операции выведите на экран.

Результат программы

```
Введите количество вершин графа: 4
=== ЗАДАНИЕ 1 ===
Матрица смежности G1:
1 1 1 0
1 0 1 0
1 1 0 1
0 0 1 1

Матрица смежности G2:
1 1 1 1
1 0 1 1
1 1 1 1
1 1 1 0
```

Рисунок 1 – генерация матриц смежности

```

=== ЗАДАНИЕ 2 ===

--- ОТОЖДЕСТВЛЕНИЕ ВЕРШИН ---
Для графа G1 - введите две вершины для отождествления (1-4): 1 2
Результат отождествления вершин 1 и 2 в G1:
0 1 1
1 1 0
1 0 1

Для графа G2 - введите две вершины для отождествления (1-4): 1 2
Результат отождествления вершин 1 и 2 в G2:
1 1 1
1 0 1
1 1 1

```

Рисунок 2 – Отождествление вершин

```

--- СЯГИВАНИЕ РЕБРА ---
Для графа G1 - введите две вершины для стягивания ребра (1-4): 1 2
Результат стягивания ребра между 1 и 2 в G1:
0 1 1
1 1 0
1 0 1

Для графа G2 - введите две вершины для стягивания ребра (1-4): 1 2
Результат стягивания ребра между 1 и 2 в G2:
1 1 1
1 0 1
1 1 1

```

Рисунок 3 – Стягивание ребра

```

--- РАСЩЕПЛЕНИЕ ВЕРШИНЫ ---
Для графа G1 - введите вершину для расщепления (1-4): 1 2
Результат расщепления вершины 1 в G1:
1 1 1 0 1
1 0 1 0 1
1 1 0 1 1
0 0 1 1 0
1 1 1 0 1

Для графа G2 - введите вершину для расщепления (1-4): Результат расщепления вершины 2 в G2:
1 1 1 1 1
1 0 1 1 1
1 1 1 1 1
1 1 1 0 1
1 1 1 1 0

```

Рисунок 4 – Расщепление вершины

```
=== ЗАДАНИЕ 3 ===
Объединение G1 и G2:
1 1 1 1
1 0 1 1
1 1 1 1
1 1 1 1

Пересечение G1 и G2:
1 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0

Кольцевая сумма G1 и G2:
0 0 0 1
0 0 0 1
0 0 1 0
1 1 0 1
```

Рисунок 5 – Объединение, пересечение и кольцевая сумма

Листинг

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <locale.h>

using namespace std;

int** generate_adjacency_matrix(int n) {
    int** matrix = new int* [n];
    for (int i = 0; i < n; i++) {
        matrix[i] = new int[n];
    }

    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            if (i == j) {
                matrix[i][j] = 0;
            }
            else {
                matrix[i][j] = rand() % 2;
                matrix[j][i] = matrix[i][j];
            }
        }
    }

    return matrix;
}

void print_matrix(int** matrix, int n) {
    cout << " ";
    for (int i = 0; i < n; i++) {
        cout << i + 1 << " ";
    }
    cout << endl;

    for (int i = 0; i < n; i++) {
        cout << i + 1 << " ";
        for (int j = 0; j < n; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }
    cout << endl;
}

void free_matrix(int** matrix, int n) {
    for (int i = 0; i < n; i++) {
        delete[] matrix[i];
    }
    delete[] matrix;
}

// Задание 2а: Отождествление вершин
int** identify_vertices(int** adj, int n, int u, int v, int& new_n) {
    new_n = n - 1;
    int** new_adj = new int* [new_n];
    for (int i = 0; i < new_n; i++) {
        new_adj[i] = new int[new_n]();
    }

    int* index_map = new int[n];
    int new_idx = 0;

    for (int i = 0; i < n; i++) {
        if (i != u && i != v) {
            index_map[i] = new_idx++;
        }
    }
}
```

```

    }
}

index_map[u] = new_n - 1;
index_map[v] = new_n - 1;

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        int new_i = index_map[i];
        int new_j = index_map[j];

        if (adj[i][j] == 1) {
            new_adj[new_i][new_j] = 1;
        }
    }
}

for (int i = 0; i < new_n; i++) {
    new_adj[i][i] = 0;
}

delete[] index_map;
return new_adj;
}

// Задание 2б: Стягивание ребра
int** contract_edge(int** adj, int n, int u, int v, int& new_n) {
    // Проверяем, существует ли ребро
    if (adj[u][v] == 0) {
        cout << "Ребро между вершинами " << u + 1 << " и " << v + 1 << " отсутствует!" << endl;
        new_n = n;
        int** copy = new int* [n];
        for (int i = 0; i < n; i++) {
            copy[i] = new int[n];
            for (int j = 0; j < n; j++) {
                copy[i][j] = adj[i][j];
            }
        }
        return copy;
    }

    return identify_vertices(adj, n, u, v, new_n);
}

int** split_vertex(int** adj, int n, int u, int& new_n) {
    new_n = n + 1;
    int** new_adj = new int* [new_n];
    for (int i = 0; i < new_n; i++) {
        new_adj[i] = new int[new_n]();
    }

    int new_vertex = n;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            new_adj[i][j] = adj[i][j];
        }
    }

    new_adj[u][new_vertex] = 1;
    new_adj[new_vertex][u] = 1;

    for (int i = 0; i < n; i++) {
        if (adj[u][i] == 1 && i != u) {

            if (rand() % 2 == 0) {

                new_adj[u][i] = 1;
                new_adj[i][u] = 1;

                new_adj[new_vertex][i] = 0;
            }
        }
    }
}

```

```

        new_adj[i][new_vertex] = 0;
    }
    else {
        new_adj[u][i] = 0;
        new_adj[i][u] = 0;
        new_adj[new_vertex][i] = 1;
        new_adj[i][new_vertex] = 1;
    }
}
}

return new_adj;
}

// Задание 3а: Объединение графов
int** union_graphs(int** adj1, int** adj2, int n) {
    int** result = new int*[n];
    for (int i = 0; i < n; i++) {
        result[i] = new int[n];
        for (int j = 0; j < n; j++) {
            result[i][j] = (adj1[i][j] || adj2[i][j]) ? 1 : 0;
        }
    }
    return result;
}

// Задание 3б: Пересечение графов
int** intersect_graphs(int** adj1, int** adj2, int n) {
    int** result = new int*[n];
    for (int i = 0; i < n; i++) {
        result[i] = new int[n];
        for (int j = 0; j < n; j++) {
            result[i][j] = (adj1[i][j] && adj2[i][j]) ? 1 : 0;
        }
    }
    return result;
}

// Задание 3в: Кольцевая сумма графов
int** ring_sum_graphs(int** adj1, int** adj2, int n) {
    int** result = new int*[n];
    for (int i = 0; i < n; i++) {
        result[i] = new int[n];
        for (int j = 0; j < n; j++) {
            result[i][j] = (adj1[i][j] != adj2[i][j]) ? 1 : 0;
        }
    }
    return result;
}

int main() {
    setlocale(LC_ALL, "Russian");
    srand(time(NULL));

    int n;
    cout << "Введите количество вершин графа: ";
    cin >> n;

    // Задание 1
    cout << "\n=== ЗАДАНИЕ 1 ===" << endl;

    int** M1 = generate_adjacency_matrix(n);
    int** M2 = generate_adjacency_matrix(n);

    cout << "\nМатрица смежности M1 (граф G1):" << endl;
    print_matrix(M1, n);

    cout << "\nМатрица смежности M2 (граф G2):" << endl;
    print_matrix(M2, n);
}

```

// Задание 2

cout << "\n=== ЗАДАНИЕ 2 ===" << endl;

// а) Отождествление вершин

cout << "\na) Отождествление вершин" << endl;

int u, v;

cout << "Для графа G1 введите номера двух вершин для отождествления (1-" << n << "): ";

cin >> u >> v;

int new_n1;

int** identify_result = identify_vertices(M1, n, u - 1, v - 1, new_n1);

cout << "Результат отождествления вершин " << u << " и " << v << " в G1:" << endl;

print_matrix(identify_result, new_n1);

// б) Стягивание ребра

cout << "\nb) Стягивание ребра" << endl;

cout << "Для графа G1 введите номера двух вершин для стягивания ребра (1-" << n << "): ";

cin >> u >> v;

int new_n2;

int** contract_result = contract_edge(M1, n, u - 1, v - 1, new_n2);

cout << "Результат стягивания ребра между " << u << " и " << v << " в G1:" << endl;

print_matrix(contract_result, new_n2);

// в) Расщепление вершины

cout << "\nv) Расщепление вершины" << endl;

cout << "Для графа G1 введите номер вершины для расщепления (1-" << n << "): ";

cin >> u;

int new_n3;

int** split_result = split_vertex(M1, n, u - 1, new_n3);

cout << "Результат расщепления вершины " << u << " в G1:" << endl;

print_matrix(split_result, new_n3);

// Задание 3

cout << "\n=== ЗАДАНИЕ 3 ===" << endl;

// а) Объединение

cout << "\na) Объединение G1 $\cup$ G2:" << endl;

int** union_result = union_graphs(M1, M2, n);

print_matrix(union_result, n);

// б) Пересечение

cout << "\nb) Пересечение G1 $\cap$ G2:" << endl;

int** intersect_result = intersect_graphs(M1, M2, n);

print_matrix(intersect_result, n);

// в) Кольцевая сумма

cout << "\nv) Кольцевая сумма G1 $\oplus$ G2:" << endl;

int** ring_sum_result = ring_sum_graphs(M1, M2, n);

print_matrix(ring_sum_result, n);

free_matrix(M1, n);

free_matrix(M2, n);

free_matrix(identify_result, new_n1);

free_matrix(contract_result, new_n2);

free_matrix(split_result, new_n3);

free_matrix(union_result, n);

free_matrix(intersect_result, n);

free_matrix(ring_sum_result, n);

return 0;

}

Вывод

В ходе выполнения лабораторной работы была разработана программа для выполнения заданий Лабораторной работы №6. В процессе выполнения работы были использованы знания о унарных и бинарных операций над графами.

Ссылка на удаленный репозиторий GitHub:

<https://github.com/MakarMn/LiOAvIZ>